
Smart Power Usage Tracker

Complete Project Tutorial

A practical guide to installing, running, using, and understanding this Django household electricity tracker.

Audience: project users, students, and developers who want to understand the complete MVP.

Contents

1. What the project does
2. Technology and architecture
3. Prepare your computer
4. Configure and start the project
5. Use the web application
6. Understand the calculations
7. Understand the database
8. Explore the REST API
9. Develop and extend the project
10. Troubleshooting and current limitations

1. What the project does

Smart Power Usage Tracker stores a household's homes, appliances, and cumulative electricity meter readings. It presents two different views of energy:

- **Actual household usage** is derived from changes between cumulative meter readings.
- **Estimated appliance usage** is calculated from appliance power, average operating hours, period length, and quantity.

Keeping these values separate is central to the application. Appliance estimates are planning estimates; they do not replace readings from the utility meter. The project also contains tariff tier calculations, a dashboard, deterministic insight rules, and authenticated JSON endpoints.

Main user journey

1. Create an account and sign in.
2. Add a home.
3. Add appliances and enter their wattage and typical usage.
4. Record at least two cumulative meter readings on different dates, or import them from CSV.
5. Review the dashboard, analytics, and insights.
6. Configure tariffs through Django Admin when you want to inspect tariff cost calculations.

2. Technology and architecture

The server is written in Python with Django and Django REST Framework. PostgreSQL stores the data. Django templates render the web pages; Chart.js draws dashboard charts. `python-dotenv` reads local environment configuration and WhiteNoise middleware is included for static files.

App	Responsibility
accounts	Registration, login, logout, profile
homes	Homes owned by signed-in users
appliances	Appliance assumptions and kWh estimates
electricity	Meter readings and CSV import
billing	Tariff tiers and cost calculation
analytics	Usage summaries for pages and dashboard
insights	Rule-based recommendations
api	REST authentication and JSON resources
config	Django settings and root URL routing

The code is organized into three broad layers: templates and views present pages, service modules hold calculations, and Django models/ORM represent and store records. Most calculation logic is in `analytics/services.py`, `billing/services.py`, and `insights/services.py`.

3. Prepare your computer

Requirements

- Python 3.12 or newer (as specified in `docs/setup.md`)
- PostgreSQL server and client tools
- Git (optional, useful for source control)
- A terminal

The packages in `requirements.txt` include Django, Django REST Framework, PostgreSQL's `psycopg2` driver, `python-dotenv`, and `WhiteNoise`.

Get the project

If you already opened this repository, skip this step. Otherwise clone or copy the repository, then change to its root directory (the directory containing `manage.py`).

```
git clone <repository-url>
cd smart-power
```

Create a virtual environment

A virtual environment keeps this project's Python packages separate from other projects.

```
python3 -m venv .venv
source .venv/bin/activate
```

On Windows PowerShell, activate it with:

```
.venv\Scripts\Activate.ps1
```

When active, your shell prompt usually shows `(.venv)`. If `python` points to a system interpreter, use `.venv/bin/python` (or `.venv\Scripts\python.exe` on Windows).

Install dependencies

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Create the PostgreSQL database

Create a database named `smart_power`, or set a different name with `DB_NAME` in `.env`. For a local PostgreSQL installation, a common setup is:

```
createdb smart_power
```

If that command is not available, create the database using `psql` or a PostgreSQL administration tool. Ensure the database user has permission to connect and create tables. The repository's sample `.env.example` assumes local peer authentication for the `john` database user; change the values for your machine.

4. Configure and start the project

Set environment variables

Copy the sample file and edit it for your local database:

```
cp .env.example .env
```

Example local settings:

```
DEBUG=True
SECRET_KEY=replace-with-a-local-development-secret
ALLOWED_HOSTS=localhost,127.0.0.1
DB_NAME=smart_power
DB_USER=your_postgres_user
DB_PASSWORD=
DB_HOST=localhost
DB_PORT=5432
CSRF_TRUSTED_ORIGINS=http://localhost:8000,http://127.0.0.1:8000
```

An empty `DB_HOST` can work with PostgreSQL's local socket and peer authentication on Linux. Use `localhost` plus the correct password for a typical password-authenticated installation. Keep `.env` private; do not commit production secrets.

Apply database migrations

Migrations create the database tables represented by the Django models.

```
python manage.py migrate
```

Optional: create an administrator account for Django Admin:

```
python manage.py createsuperuser
```

Start the development server

```
python manage.py runserver 127.0.0.1:8000
```

Open <http://127.0.0.1:8000/>. The root page redirects to the dashboard. Use `Ctrl+C` to stop the server. The development server is intended for local development, not production hosting.

Useful commands

<code>python manage.py check</code>	<code># inspect Django configuration</code>
<code>python manage.py showmigrations</code>	<code># see migration status</code>
<code>python manage.py makemigrations</code>	<code># generate migrations after model changes</code>
<code>python manage.py migrate</code>	<code># apply migrations</code>
<code>python manage.py createsuperuser</code>	<code># create an admin login</code>
<code>python manage.py test</code>	<code># run the project's automated tests</code>

5. Use the web application

Create an account and sign in

Visit `/accounts/register/`, provide your name, username, email, and password, then submit. The application signs a new user in and sends them to the dashboard. Existing users can sign in at `/accounts/login/`. Pages for household data require authentication.

Add a home

Open `/homes/` and create a home with a name. A home is the grouping boundary for appliances and meter readings. Each home belongs to the account that created it. The first home is selected by dashboard and list pages in the current implementation.

Add appliances

Go to `/appliances/` or use the home detail page to add a device. Enter:

- A recognizable name and category
- Rated power in watts
- Average usage hours per day
- Days in the estimate period (defaults to 30)
- Quantity (defaults to 1)
- Whether it is active

Only active appliances are included in the combined estimate and breakdown. The `standby_power_watts` value is saved, but the current estimation formula does not include standby usage. Appliance values are assumptions; improve their accuracy with actual nameplate ratings and realistic usage hours.

Record meter readings

Open `/readings/` and add readings with a date and cumulative meter value in kWh. Enter the number shown on the meter, not the electricity used since the previous reading. For example, if the meter shows 1,245.6 kWh now and 1,310.1 kWh later, the period usage is 64.5 kWh.

You need at least two readings to calculate a consumption difference. Dates should be in sequence and meter values should normally rise. If a meter is replaced or reset and its value drops, the current analytics skips that decreasing interval; it does not infer the replacement meter's starting value. Keep notes and consult the actual meter history when interpreting such cases.

Import readings from CSV

The reading form supports a CSV upload with the exact required column names `date` and `kwh`:

```
date,kwh
2026-01-01,1245.600
2026-02-01,1310.100
2026-03-01,1372.400
```

Dates must be valid dates, values numeric and non-negative. The import labels records with source `csv_import`. The application validates rows before saving the imported set. Use cumulative meter values in the `kwh` column, not interval usage values.

Read the dashboard and insights

The dashboard at `/dashboard/` shows the current (first) home, actual usage, estimated appliance usage, active appliance count, and highest estimated appliance. The line chart shows daily usage inferred from readings; the appliance chart ranks estimates. The comparison chart places the two totals side by side, so compare their time spans carefully before drawing conclusions.

The insights page (`/insights/`) presents deterministic rules. Current examples flag a large combined appliance estimate, actual usage more than 20% above the estimate, or an appliance rated above 1,500 W. These are prompts to investigate, not diagnoses or automated measurements. Dashboard visits rebuild the insight rows for the selected home.

Review tariffs

Tariffs and tariff tiers are Django Admin-managed records. Sign in at `/admin/` as a superuser to create or edit them. A tier has a minimum kWh, optional maximum kWh, and rate per kWh. The billing page shows a sample calculation for 142 kWh using the first tariff in the tariff list. Configure tier ranges deliberately; the implementation does not enforce that tiers are contiguous or non-overlapping.

6. Understand the calculations

Appliance estimate

The model calculates:

```
estimated kWh = (power watts × hours per day × days in period × quantity) / 1000
```

Example: a 100 W device used 5 hours/day for 30 days, with quantity 2:

```
(100 × 5 × 30 × 2) / 1000 = 30 kWh
```

This is an estimate for the configured period, not automatically a monthly bill. If you set `days_per_period` to 7, the result covers seven days. The combined estimate sums active appliances.

Metered household usage

Meter entries are cumulative counter values. The service sorts readings by date and adds each non-negative difference between consecutive values:

```
period use = current cumulative reading - previous cumulative reading  
total actual use = sum of valid period use values
```

The first reading alone contributes no consumption because there is no earlier reading for comparison. If a later reading falls below the previous value, analytics ignores that interval and continues from the new reading for the next interval. Enter correct dates and meter history to avoid misleading totals.

Tiered tariff cost

For each configured tariff tier, the service charges the portion of total consumption above that tier's minimum, up to the tier's maximum (or all remaining consumption for an open-ended tier). Charges are summed and rounded to two decimal places using half-up rounding. Example: configure tiers 0–50 kWh at 1.00 and 50+ kWh at 2.00; a 70 kWh usage gives $50 \times 1 + 20 \times 2 = 90.00$ currency units. The project does not assign a currency symbol or implement taxes, fixed fees, or utility-specific billing rules.

7. Understand the database

- **User → Home:** one account can own multiple homes.
- **Home → Appliance:** each appliance belongs to a home; deleting the home deletes its appliances.
- **Home → MeterReading:** each reading belongs to a home; deleting the home deletes those readings.
- **Tariff → TariffTier:** each tariff can have multiple ordered tiers.
- **Home → Insight:** generated recommendations belong to a home.

Models hold persistent data and basic validation. `full_clean()` is explicitly called by the main appliance, reading, and CSV web forms before saving. Ownership is checked in protected web views and API querysets so reads are filtered to the signed-in user's homes. The user/home relationship is implemented with Django's built-in user model.

8. Explore the REST API

The API is rooted at `/api/` and uses Django REST Framework session authentication (and Basic authentication is configured globally). The default API permission requires an authenticated user, with registration and login endpoints explicitly open.

Authentication endpoints

Method	Endpoint	Purpose
POST	<code>/api/auth/register/</code>	Create account and sign in
POST	<code>/api/auth/login/</code>	Sign in
POST	<code>/api/auth/logout/</code>	Sign out
GET	<code>/api/auth/me/</code>	Return current username and email

Data endpoints

Method	Endpoint	Purpose
GET, POST	<code>/api/homes/</code>	List or create homes
GET, PUT, PATCH, DELETE	<code>/api/homes/{id}/</code>	Retrieve, change, or delete a home
GET, POST	<code>/api/appliances/</code>	List or create appliances
GET, PUT, PATCH, DELETE	<code>/api/appliances/{id}/</code>	Retrieve, change, or delete an appliance
GET, POST	<code>/api/readings/</code>	List or create meter readings
GET, PUT, PATCH, DELETE	<code>/api/readings/{id}/</code>	Retrieve, change, or delete a reading
GET	<code>/api/insights/</code>	List insights
GET	<code>/api/dashboard/</code>	Dashboard summary for first home
GET	<code>/api/analytics/daily/</code>	Daily usage data for first home
GET	<code>/api/analytics/actual-vs-estimated/</code>	Compare totals for first home

DRF's model viewsets also provide the usual detail actions for the resources. Use an API client such as `curl`, Postman, or the browser's developer tools. For session authentication, first log in and preserve the session cookie; unsafe requests also need a CSRF token. API JSON date values should use ISO format (YYYY-MM-DD). Resource querysets are filtered by ownership.

Implementation status: the weekly and monthly analytics routes exist, but currently return an empty data array. Tariffs have serializers in the code, but no tariff API route is registered. The API offers reading CRUD, not CSV upload. The API dashboard/analytics endpoints choose the user's first home.

9. Develop and extend the project

Find the relevant files

- Add a database field in the corresponding app's `models.py`, then generate and apply a migration.
- Put reusable calculations in the relevant `services.py` module.
- Update page behavior in the app's `views.py` and route it in `urls.py`.
- Update API payloads in `api/serializers.py` and API behavior in `api/views.py`.
- Edit page templates under `templates/`.
- Django settings and root route includes live in `config/settings.py` and `config/urls.py`.

Typical model change workflow

1. Edit the model in the owning app.
2. Run `python manage.py makemigrations <app_name>`.
3. Review the generated file in `<app_name>/migrations/`.
4. Run `python manage.py migrate`.
5. Update forms, serializers, templates, and tests that depend on the changed field.

Keep actual usage and appliance estimates visibly distinct. Keep business calculations in services so web and API code can share them. Use ownership-filtered querysets whenever handling a user-supplied object ID.

10. Troubleshooting and current limitations

Database connection fails

Confirm PostgreSQL is running, the database exists, and `.env` has the right `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, and `DB_PORT`. Try connecting with `psql` using the same database name and user. On local Linux, an empty host often selects a Unix socket and peer authentication; `localhost` usually selects TCP and password rules.

ModuleNotFoundError after installation

Activate `.venv`, then run `pip install -r requirements.txt` using that environment's Python. Check with `python -m pip --version`.

DisallowedHost error

Add the hostname you used to `ALLOWED_HOSTS` in `.env`, restart the development server, and reload.

Dashboard has zero actual usage

Add two valid cumulative readings. One reading has no interval, and a decrease is skipped in analytics. Check that the readings belong to the currently selected first home.

Project scope and caveats

This repository is a development-ready MVP, not a production-hardened utility billing system. It has no live utility/EEU integration, no ML forecasting, no smart meter connection, and no mobile client. The current browser UI uses the first home for appliance, reading, dashboard, and analytics lists even though the data model supports multiple homes. Weekly and monthly API analytics are placeholders. Tariff rules are illustrative and should not be treated as an official bill. For production, configure `DEBUG=False`, use a strong secret and secure database credentials, serve over HTTPS, configure static file hosting and backups, and review security settings for the deployment environment.

Project file map

<code>config/</code>	Django project settings and URL routing
<code>accounts/</code>	Account views and routes
<code>homes/</code>	Home model and pages
<code>appliances/</code>	Appliance model, estimates, and pages
<code>electricity/</code>	Meter model, entry, and CSV import
<code>billing/</code>	Tariff models and tier pricing service
<code>analytics/</code>	Usage services and dashboard pages
<code>insights/</code>	Recommendation rules and pages
<code>api/</code>	Serializers, API views, and routes
<code>templates/</code>	Django HTML templates
<code>static/</code>	Static assets
<code>docs/</code>	Project documentation, including this tutorial
<code>manage.py</code>	Django management command entry point
<code>requirements.txt</code>	Python dependencies

Next steps

Start with one home, add a few known appliances, then enter at least two dated cumulative meter readings. Compare the separate actual and estimated figures, inspect the formula inputs, and use the code map above to trace a number from its model/service to its page or API response.